RENEWABLE ENERGY MONITORING SYSTEM

A SQL-Based Database Project

Submitted by: Praneet Biswal

Roll Number: 2306351

Batch / Program: B.Tech (Information Technology)

Date: April 2026

1. Problem Statement

The increasing adoption of renewable energy sources such as solar, wind, and hydro requires efficient monitoring systems. Managing energy production data across multiple locations can be complex and unstructured without a proper data management solution.

This project aims to design a relational database system to store, retrieve, and analyze renewable energy production data using SQL. The goal is to simplify data management and enable efficient querying across plants and energy types.

1.1 Context

India and many other countries are rapidly scaling up renewable energy capacity. Reliable data management infrastructure is essential to track production metrics, compare performance across sites, and support data-driven decisions.

1.2 Objectives

- Design a normalized relational database schema for renewable energy data.
- Implement SQL tables with appropriate primary and foreign key constraints.
- Populate the database with sample production records across locations.
- Demonstrate querying capabilities including filtering, aggregation, and joins.

2. Solution & Features

The proposed system provides a structured relational database to manage renewable energy data efficiently. It uses three normalized tables connected through foreign key relationships, enabling powerful querying and aggregation.

2.1 Database Schema

Table	Columns	Description
energy_sources	source_id, source_name	Stores energy types (Solar, Wind, Hydro)
locations	location_id, location_name, city	Stores plant locations across cities
energy_production	production_id, source_id, location_id, production_date, energy_generated_kwh	Records daily production with FK links

2.2 Key Features

- Stores multiple energy source types — Solar, Wind, and Hydro.
- Tracks energy generation volumes across different plant locations.
- Enables row-level filtering using the WHERE clause.
- Performs aggregation and total computation using GROUP BY.
- Sorts records in ascending or descending order using ORDER BY.
- Combines data across multiple tables using JOIN operations.
- Normalized design avoids data redundancy and ensures integrity.

2.3 SQL Concepts Demonstrated

SQL Clause	Purpose	Example Use
SELECT	Retrieve records	Fetch all production entries
WHERE	Filter rows	Records where kWh > 500
GROUP BY	Aggregate by category	Total energy per source type
ORDER BY	Sort results	Descending by energy generated
JOIN	Combine multiple tables	Source name + location + output

3. SQL Implementation

3.1 Table Creation

```
CREATE TABLE energy_sources (
    source_id INT PRIMARY KEY,
    source_name VARCHAR(50)
);

CREATE TABLE locations (
    location_id INT PRIMARY KEY,
    location_name VARCHAR(100),
    city VARCHAR(50)
);

CREATE TABLE energy_production (
    production_id INT PRIMARY KEY,
    source_id INT,
    location_id INT,
    production_date DATE,
    energy_generated_kwh FLOAT,
    FOREIGN KEY (source_id) REFERENCES energy_sources(source_id),
    FOREIGN KEY (location_id) REFERENCES locations(location_id)
);
```

3.2 JOIN Query

```
SELECT
    es.source_name,
    l.location_name,
    ep.production_date,
    ep.energy_generated_kwh
FROM energy_production ep
JOIN energy_sources es ON ep.source_id = es.source_id
JOIN locations l ON ep.location_id = l.location_id;
```

3.3 Screenshots

table_creation.png
Table creation — all three tables created successfully in MySQL.

screenshot: table_creation.png

MySQL 8.0 — renewable_energy_db

```
mysql> CREATE DATABASE renewable_energy_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE renewable_energy_db;
Database changed

mysql> CREATE TABLE energy_sources (
    source_id INT PRIMARY KEY,
    source_name VARCHAR(50)
);
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE locations (
    location_id INT PRIMARY KEY,
    location_name VARCHAR(100),
    city VARCHAR(50)
);
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE energy_production (
    production_id INT PRIMARY KEY,
    source_id INT,
    location_id INT,
    production_date DATE,
    energy_generated_kwh FLOAT,
    FOREIGN KEY (source_id) REFERENCES energy_sources(source_id),
    FOREIGN KEY (location_id) REFERENCES locations(location_id)
);
Query OK, 0 rows affected (0.06 sec)

mysql> SHOW TABLES;

+--------------------------+
| Tables_in_renewable_energy_db |
+--------------------------+
| energy_production |
| energy_sources |
| locations |
+--------------------------+
3 rows in set (0.00 sec)

mysql>
```

data_output.png

*Data output — SELECT * results showing all inserted records.*

screenshot: data_output.png

MySQL 8.0 — renewable_energy_db

```
mysql> INSERT INTO energy_sources VALUES (1,'Solar'),(2,'Wind'),(3,'Hydro');
Query OK, 3 rows affected (0.02 sec)

mysql> INSERT INTO locations VALUES (1,'Plant A','Delhi'),(2,'Plant B','Mumbai'),(3,'Plant C','Chennai');
Query OK, 3 rows affected (0.01 sec)

mysql> INSERT INTO energy_production VALUES
   (1,1,1,'2026-04-01',500.5),(2,2,1,'2026-04-01',300.2),
   (3,1,2,'2026-04-02',450.0),(4,3,3,'2026-04-02',800.0),(5,2,2,'2026-04-03',600.3);
Query OK, 5 rows affected (0.02 sec)

mysql> SELECT * FROM energy_production;

+---------------+-----------+-------------+----------------+----------------------+
| production_id | source_id | location_id | production_date | energy_generated_kwh |
+---------------+-----------+-------------+----------------+----------------------+
|             1 |         1 |           1 | 2026-04-01     |                500.5 |
|             2 |         2 |           1 | 2026-04-01     |                300.2 |
|             3 |         1 |           2 | 2026-04-02     |                450   |
|             4 |         3 |           3 | 2026-04-02     |                  800 |
|             5 |         2 |           2 | 2026-04-03     |                600.3 |
+---------------+-----------+-------------+----------------+----------------------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM energy_sources;

+-----------+------------+
| source_id | source_name |
+-----------+------------+
|         1 | Solar      |
|         2 | Wind       |
|         3 | Hydro      |
+-----------+------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM locations;

+-------------+---------------+---------+
| location_id | location_name | city    |
+-------------+---------------+---------+
|           1 | Plant A       | Delhi   |
|           2 | Plant B       | Mumbai  |
|           3 | Plant C       | Chennai |
+-------------+---------------+---------+
3 rows in set (0.00 sec)

mysql>
```

groupby.png
GROUP BY result — total energy aggregated per source type.

screenshot: groupby.png

MySQL 8.0 — renewable_energy_db

```
mysql> SELECT * FROM energy_production WHERE energy_generated_kwh > 500;

+---------------+-----------+-------------+----------------+----------------------+
| production_id | source_id | location_id | production_date | energy_generated_kwh |
+---------------+-----------+-------------+----------------+----------------------+
|             1 |         1 |           1 | 2026-04-01     |                500.5 |
|             4 |         3 |           3 | 2026-04-02     |                  800 |
|             5 |         2 |           2 | 2026-04-03     |                600.3 |
+---------------+-----------+-------------+----------------+----------------------+
3 rows in set (0.00 sec)

mysql> SELECT source_id, SUM(energy_generated_kwh) AS total_energy
   FROM energy_production GROUP BY source_id;

+-----------+--------------+
| source_id | total_energy |
+-----------+--------------+
|         1 |        950.5 |
|         2 |        900.5 |
|         3 |          800 |
+-----------+--------------+
3 rows in set (0.01 sec)

mysql> SELECT * FROM energy_production ORDER BY energy_generated_kwh DESC;

+---------------+-----------+-------------+----------------+----------------------+
| production_id | source_id | location_id | production_date | energy_generated_kwh |
+---------------+-----------+-------------+----------------+----------------------+
|             4 |         3 |           3 | 2026-04-02     |                  800 |
|             5 |         2 |           2 | 2026-04-03     |                600.3 |
|             1 |         1 |           1 | 2026-04-01     |                500.5 |
|             3 |         1 |           2 | 2026-04-02     |                450   |
|             2 |         2 |           1 | 2026-04-01     |                300.2 |
+---------------+-----------+-------------+----------------+----------------------+
5 rows in set (0.00 sec)

mysql>
```

join.png

JOIN result — source names and locations resolved across tables.

screenshot: join.png

MySQL 8.0 — renewable_energy_db

```
mysql> SELECT
   es.source_name,
   l.location_name,
   ep.production_date,
   ep.energy_generated_kwh
FROM energy_production ep
JOIN energy_sources es ON ep.source_id = es.source_id
JOIN locations l ON ep.location_id = l.location_id;

+-------------+---------------+----------------+----------------------+
| source_name | location_name | production_date | energy_generated_kwh |
+-------------+---------------+----------------+----------------------+
| Solar       | Plant A       | 2026-04-01     |                500.5 |
| Wind        | Plant A       | 2026-04-01     |                300.2 |
| Solar       | Plant B       | 2026-04-02     |                450   |
| Hydro       | Plant C       | 2026-04-02     |                  800 |
| Wind        | Plant B       | 2026-04-03     |                600.3 |
+-------------+---------------+----------------+----------------------+
5 rows in set (0.02 sec)

mysql>
```

4. Tech Stack

Technology	Version / Details	Role
MySQL	8.0+	Primary relational database engine
SQL	Standard SQL	Query language for all operations
DB Design	Normalization (3NF)	Schema design principle applied

5. Unique Points

- Simple and fully normalized database design following 3NF principles.
- Covers all essential SQL operations in a single cohesive project.
- Foreign key constraints enforce data integrity at the database level.
- Easy to extend — new energy sources or locations require only new INSERT rows.
- Lightweight with no external dependencies beyond a standard MySQL installation.

6. Future Improvements

IoT Integration: Connect real-time sensor feeds to auto-populate the production table.

Web Dashboard: Build a visualization layer using Python (Flask/Django) or Node.js.

Cloud Deployment: Host on AWS RDS or Google Cloud SQL for scalable, remote access.

ML / Analytics: Apply Python-based analytics to forecast energy production trends.

User Authentication: Add role-based access control for operators and administrators.

Mobile App: Provide a lightweight mobile interface for field monitoring.